

Quaternionen-Zerlegung

Zerlegung natürlicher Zahlen in Prim-Quaternionen im Hurwitz-Quaternionen-Ring

SageMath 10.8

13. März 2026

Zusammenfassung

Dieser Artikel beschreibt die Zerlegung natürlicher Zahlen in ein Produkt von Prim-Quaternionen im Hurwitz-Quaternionen-Ring $\mathcal{H}$. Ausgehend vom Satz von Lagrange (jede natürliche Zahl ist Summe von vier Quadraten) wird ein Algorithmus vorgestellt, der zu gegebenem $n \in \mathbb{N}$ eine Faktorisierung in Prim-Quaternionen $\pi_1, \dots, \pi_k$ mit $N(\pi_1) \cdots N(\pi_k) = n$ liefert. Die Implementierung erfolgt in SageMath.

1 Import und Hurwitz-Quaternionen-Ring

Der Hurwitz-Quaternionen-Ring $\mathcal{H}$ ist die Quaternionen-Algebra über $\mathbb{Q}$ mit den Parametern $(-1, -1)$, d. h. $i^2 = j^2 = k^2 = ijk = -1$ (Hamilton). In SageMath wird er definiert durch:

```
1 from sage.all import *
2 from sage.arith.misc import four_squares
3
4 # Definition des Hurwitz-Quaternionen-Rings über  $\mathbb{Q}$ 
5 H = QuaternionAlgebra(QQ, -1, -1)
6 H
```

Ausgabe: Quaternion Algebra (-1, -1) with base ring Rational Field

2 Funktion: Zerlegung in Prim-Quaternionen

Die **Quaternionen-Norm** eines Elements $q = a + bi + cj + dk$ ist

$$N(q) = a^2 + b^2 + c^2 + d^2.$$

Nach dem **Satz von Lagrange** ist jede natürliche Zahl n Summe von vier Quadraten: $n = a^2 + b^2 + c^2 + d^2$. Somit existiert stets ein Quaternion $q \in \mathcal{H}$ mit $N(q) = n$.

Die Funktion `quaternions_zerlegung(n)` zerlegt n in eine Kette von Prim-Quaternionen, indem für jeden Primfaktor p von n ein Quaternion π mit $N(\pi) = p$ gefunden wird (ebenfalls über `four_squares`).

```
1 def quaternions_zerlegung(n):
2     """
3     Zerlegt eine natürliche Zahl n in ein Produkt von Prim-Quaternionen.
4     Basiert auf dem Hurwitz-Quaternionen-Ring (H).
5     """
6     # Wir suchen ein Quaternion q, dessen Norm n ist
7     # (Satz von Lagrange: Jede natürliche Zahl ist Summe von 4 Quadraten)
8     coords = four_squares(n)
```

```

9      q = H(coords)
10
11      print(f"### #Energiedoku: Analyse von n = {n}")
12      print(f"Basis-Quaternion q: {q}")
13      print(f"Norm N(q): {q.reduced_norm()}")
14      print("-" * 30)
15
16      # Zerlegung der Norm n in Primfaktoren p_i
17      # Für jedes p_i ein zugehöriges Prim-Quaternion pi_i
18      faktoren_n = factor(n)
19      prim_quaternionen_kette = []
20
21      for p, expo in faktoren_n:
22          p_coords = four_squares(p)
23          pi = H(p_coords)
24          for _ in range(expo):
25              prim_quaternionen_kette.append(pi)
26
27      return prim_quaternionen_kette

```

3 Beispiel

Für $n = 110567868768768768767$ erhält man:

```

1  n_test = 110567868768768768767
2  kette = quaternions_zerlegung(n_test)
3
4  print("Zerlegung in Prim-Quadrupel (Familien-Komponenten):")
5  for i, pi in enumerate(kette):
6      print(f"Faktor {i+1} (Prim-Quaternion): {pi} | Norm: {pi.reduced_norm()}")

```

Ausgabe:

```

### #Energiedoku: Analyse von n = 110567868768768768767
Basis-Quaternion q: 66 + 1511*i + 92589*j + 10515125713*k
Norm N(q): 110567868768768768767
-----
Zerlegung in Prim-Quadrupel (Familien-Komponenten):
Faktor 1 (Prim-Quaternion): 3 + 5*i + 177*j + 16188*k | Norm: 262082707
Faktor 2 (Prim-Quaternion): 1 + 20*i + 402*j + 649524*k | Norm: 421881588581

```

4 Verifikation

Das Produkt der Normen der Prim-Quaternionen muss wieder n ergeben:

```

1  check_norm = prod(p.reduced_norm() for p in kette)
2  print("-" * 30)
3  print(f"Verifikation der Gesamt-Energie (Produkt der Normen): {check_norm}")
4  print(f"Erwartet: {n_test} -> Korrekt: {check_norm == n_test}")

```

Ausgabe:

Verifikation der Gesamt-Energie (Produkt der Normen): 110567868768768768767
Erwartet: 110567868768768768767 -> Korrekt: True

Zusammenfassung

Die Zerlegung natürlicher Zahlen in Prim-Quaternionen im Hurwitz-Ring nutzt den Satz von Lagrange und die Faktorisierung in $\mathbb{Z}$. Für jeden Primfaktor p von n wird ein Quaternion π mit $N(\pi) = p$ gefunden; das Produkt der Normen aller Prim-Quaternionen ergibt n und bestätigt die Korrektheit der Zerlegung.